

Technical Build Spec & Screen-by-Screen Scope

MVP specification for developer quoting, scoping, and first-build execution.

Prepared for founder review | Version 1.0 | 27 March 2026

Why this matters: This document translates the RateIt concept into a practical first-build scope. It is intended to help a coder or agency price the MVP without trying to build the whole empire on day one.

1. Product Summary

RateIt is a browser extension that checks the store domain a shopper is visiting against a reviewed merchant database and displays a trust verdict with short reasons before checkout.

Item	Definition
Primary user	Online shopper visiting an unfamiliar e-commerce site
Core promise	Warn the user when trust signals are weak, missing, or risky before they buy
MVP verdicts	Trusted, Caution, High Risk, Insufficient Data
Primary surfaces	Browser extension popup, full report page, report-this-store form, admin dashboard

Why this matters: The MVP is not a full review platform, public forum, or AI detective. It is a disciplined trust-signal product at the moment of purchase.

2. MVP Boundary

Build first:

- A reviewed merchant database with domain-based records
- An admin dashboard to create, edit, assess, and reassess merchants
- A rules-based scoring engine that outputs a verdict and top reasons
- A browser extension popup that reads the current domain and displays the verdict
- A report-this-store workflow that stores incoming user reports for review

Do not build in phase one:

- Automated web scraping at scale
- Machine learning or AI scoring
- Merchant self-service portal and billing
- Public consumer forum or open review wall
- Mobile app
- Large public directory with advanced search

3. User Roles

Role	What they do	What they can access
Shopper	Installs extension, views verdicts, opens full report, submits report	Popup, full report, reporting form

Reviewer/Admin	Creates assessments, updates verdicts, reviews user reports	Admin dashboard
Lead reviewer (optional)	Approves escalated or high-risk merchant decisions	Admin dashboard + approval fields

4. System Architecture in Plain English

- Database - stores merchants, assessments, reports, verdict history, and review dates.
- Admin web app - the control room where merchants are reviewed and verdicts are managed.
- Rules engine - converts pillar ratings and red flags into a final verdict.
- API layer - serves merchant verdicts to the extension and accepts user reports.
- Browser extension - the user-facing product that appears on store pages.

Why this matters: A backend alone is not the product. The product requires code that collects, collates, interprets, and displays the data.

5. Screen Inventory

ID	Screen	User	Purpose
S1	Extension popup - known merchant	Shopper	Show verdict, pillar snapshot, top reasons, and next actions
S2	Extension popup - insufficient data	Shopper	Show unknown-state guidance and request/report actions
S3	Full report page	Shopper	Show deeper trust explanation for the current merchant
S4	Report this store form	Shopper	Capture reports tied to merchant/domain
S5	Admin login	Reviewer	Secure access to backend
S6	Merchant list/dashboard	Reviewer	Search, filter, and review merchant records
S7	Merchant assessment/detail	Reviewer	Create/edit verdicts, pillar ratings, rationale, and review dates
S8	User reports queue	Reviewer	Moderate, triage, and link incoming reports to merchants

6. Screen-by-Screen Scope

S1. Extension popup - known merchant

- Visible elements: RateIt Signal header, domain, last reviewed date, large verdict label, one-line explanation, four-pillar snapshot, 3-4 top reasons, action buttons.
- Actions: View Full Report, Report This Store, Why This Rating?
- States supported: Trusted, Caution, High Risk.
- Rules: Must load quickly; must not expose internal notes; should gracefully fail to Insufficient Data if API cannot find a record.

S2. Extension popup - insufficient data

- Visible elements: domain, neutral unknown-state verdict, short explanation, unknown pillar state, action buttons.
- Actions: Request Review or Report This Store, Why This Rating?
- Rules: Must not imply a negative verdict when evidence is absent.

S3. Full report page

- Visible elements: store name/domain, verdict, last updated, one-paragraph summary, four-pillar breakdown, positive signals, warning signals, consumer caution notes, report CTA.
- Rules: Page must be readable on desktop; generated from stored merchant record; no anonymous comment wall in MVP.

S4. Report this store form

- Fields: domain or merchant, report type, description, order/refund issue type, optional screenshot/file or URL, consent checkbox, submit.
- Rules: Submission stored with status New; duplicate submissions should still be allowed but linked to the same merchant when possible.

S5. Admin login

- Fields: email, password, sign in. Optional forgot password later.
- Rules: Must protect admin routes; no shopper access.

S6. Merchant list/dashboard

- Visible elements: search by domain/store, filters by verdict/status/category, sortable list, quick counts, create merchant button.
- Rules: Should surface merchants due for review and newly reported merchants.

S7. Merchant assessment/detail

- Fields: store identity, review scope, evidence source checkboxes, pillar ratings, positive signals, warning signals, red flags, final verdict, internal rationale, public summary, top reasons, next review date.

- Rules: Should save drafts; should track last editor and last updated date; must support review history.

S8. User reports queue

- Visible elements: report list, status, linked merchant, severity/issue type, created date, open report view, resolve/escalate actions.
- Rules: Report should be linkable to existing merchant or used to create a new merchant record.

7. Core Data Model

Entity	Required core fields	Why it exists	Notes
Merchant	id, name, domain, category, verdict, last_reviewed, status, public_summary	Master record used by extension and report pages	One merchant per primary domain in MVP
Assessment	merchant_id, pillar ratings, red flags, internal rationale, public reasons, reviewer_id, review date	Stores the reasoning behind the verdict	Should support history/versioning
User Report	merchant/domain, issue type, description, attachments/URL, created date, status	Captures consumer input	Can exist before a merchant is formally assessed
Reviewer/Admin	id, name, email, role, auth details	Secures backend access and audit trail	Simple role model is enough for MVP

8. Verdict Logic

The rules engine can be simple and deterministic in MVP.

Verdict	Minimum decision rule for MVP
Trusted	At least 3 pillars Strong, no major red-flag trigger, no severe consumer-harm pattern.
Caution	Mixed profile overall, one weak pillar or multiple medium concerns, enough uncertainty that a shopper should slow down.
High Risk	Two or more weak pillars, or one severe failure plus repeated red flags or harmful pattern.
Insufficient Data	Evidence too thin to rate fairly or merchant not yet fully assessed.

Why this matters: Do not over-engineer the first scoring model. The win is consistency and defensibility, not mathematical theatre.

9. API Scope

Endpoint / function	What it does	Used by
GET merchant by domain	Returns verdict, summary, pillar snapshot, top reasons, last reviewed	Extension popup
GET merchant full report	Returns full report data for report page	Report page
POST user report	Creates a new user report tied to merchant/domain	Report form
GET merchant list / filters	Returns merchants for admin list	Admin dashboard
GET merchant detail	Returns full assessment record for editing	Admin detail
POST/PUT merchant assessment	Creates or updates merchant assessment and verdict	Admin detail
GET user reports queue	Returns reports for moderation	Reports queue

10. Acceptance Criteria for Quote and MVP

- A reviewer can create a merchant record and save an assessment.
- The system can store pillar ratings, red flags, rationale, and public reasons.
- The extension can detect the current domain and request a verdict from the API.
- Known merchants display verdict, top reasons, and a link to the full report.
- Unknown merchants display Insufficient Data without implying guilt.
- A shopper can submit a report from the popup or report page.
- An admin can view new reports and link them to an existing or new merchant.
- All core records show last reviewed / last updated information.

11. Out of Scope for MVP

- Automated web scraping across the open internet
- AI-generated verdicts
- Merchant subscriptions and self-serve billing
- Open public review feeds or comment threads
- Native mobile apps
- Complex trust analytics dashboards for consumers

12. Suggested Build Order

1. Database schema and admin authentication

2. Merchant assessment workflow and verdict storage
3. Rules engine and public summary generation
4. API endpoints for merchant lookup and reports
5. Browser extension popup and full report page
6. User reporting flow and admin moderation queue

13. Questions the Developer Should Answer in Their Quote

- What stack do you recommend for extension + admin web app + API + database?
- How will you structure authentication and admin roles?
- How will the extension securely call the backend?
- How will you store report attachments or URLs?
- How will verdict history/versioning be handled?
- What deployment environment do you recommend for MVP?
- What is included in testing, bug fixing, and post-launch support?

14. Handoff Summary

The first build is a reviewed, rules-based trust signal system - not a giant automated scam-detection platform. If a developer can deliver the merchant database, admin workflow, rules engine, API bridge, and browser extension popup cleanly, RateIt will have a real product to test in the market.